学生视角 • 项目学习指南

边缘 AI 云台跟随系统

要学什么 · 学到什么程度 · 怎么验证

文档信息	
文档类型	项目配套学习指南（学生视角）
适用对象	有一定编程基础、想进入嵌入式 / 边缘 AI 领域的在校学生
配套项目	边缘 AI 云台跟随系统（Jetson + STM32 + 总线舵机）
知识模块	12 个模块，覆盖 8 个技术方向
学习周期	按阶段推进，约 8~12 周（每天 2~3 小时）
配套仓库	github.com/ZELDA-LINK-F/edge-ai-learning

本指南的定位：不是「教程」，而是「学习地图」——

告诉你每个知识点要学到什么深度、用什么方法验证、以及它在项目里对应哪段代码。

目 录

01 前言 —— 怎么用这份指南

02 第一章 能力模型 —— 三级能力标准：了解 / 会用 / 精通

03 第二章 知识地图 —— 12 个模块的全景图与依赖关系

04 第三章 分模块详解 —— 每个模块：知识点 + 掌握程度 + 项目对应 + 练习题

05 第四章 学习路线 —— 5 个阶段、8~12 周的具体安排

06 第五章 能力自评 —— 自测表与达标标准

07 第六章 岗位对应 —— 这些知识能对应什么工作

08 第七章 常见误区 —— 学生最容易走偏的 10 个地方

09 附录 —— 术语表、推荐资料、项目文件索引

前言：怎么用这份指南

0.1 为什么需要这样一份文档

很多学生在做项目时会陷入一种状态：跟着教程一步步做出来了，但合上电脑后说不清楚自己到底学了什么。代码能跑，但换个需求就不会改；面试被问「你这个项目难点在哪」，只能支支吾吾。

问题的根源是：**做项目 ≠ 学到知识**。项目只是载体，真正要沉淀的是背后的知识体系和解决问题的能力。

这份指南做的事情，就是把这个项目拆解成可学习、可验证、可自测的知识点，让你清楚知道：

- 我现在会什么、不会什么
- 每个知识点要学到什么深度才算「够用」
- 怎么验证自己真的学会了（而不是看懂了）
- 这些知识对应哪些岗位要求

0.2 文档的三个特点

特点	说明
从学生视角出发	不假设你有工作经验，每个概念都先讲「为什么需要它」
有明确的深度标准	每个知识点标注「了解 / 会用 / 精通」三级，避免学过头或学不够
可验证	每个模块都有练习题，做完能自己判断是否达标

0.3 建议的使用方法

不要从头读到尾。建议这样用：

步骤	做什么	目的
第 1 步	读第二章「知识地图」	先建立全局认知，知道有哪些东西要学
第 2 步	读第四章「学习路线」	选一个阶段，按顺序推进
第 3 步	每学一个模块，回到第三章看详细要求	对照知识点清单，逐条打钩
第 4 步	做完练习题	验证自己真的会了

第 5 步	用第五章的自评表打分	知道下一步该补什么

0.4 前置要求

本指南假设你已经具备以下基础（如果不具备，先补这些）：

领域	最低要求
C 语言	会用指针、结构体、数组，能读懂几百行的程序
Python	会写函数、类、读写文件，能用 pip 装包
Linux 基础	会 cd / ls / cat / chmod，能用 SSH 登录远程机器
电子基础	知道电压、电流、地（GND）的概念，认识电阻、杜邦线
数学	高中水平的线性函数、初步的统计概念（平均值、方差）

如果这些都不具备，也不要紧 —— 可以先从模块 0 开始补。

第一章　能力模型

在开始学习之前，先建立「学到什么程度算够」的判断标准。本指南采用三级能力模型：

1.1 三级能力标准

等级	含义	判断标准	占比建议
① 了解	知道这个概念存在、知道它是干什么的	能用自己的话解释它是什么，能在资料里找到它	所有知识点都要达到
② 会用	能在项目中实际使用，能改参数、能排查常见问题	能独立写出可运行的代码，出错时能定位	核心知识点必须达到
③ 精通	理解底层原理，能优化、能设计、能教别人	能解释「为什么这样设计」，能做性能优化或方案设计	选 2~3 个方向深入

重要提醒：不需要所有知识点都达到「精通」。

学生的常见误区是「什么都想学精」，结果每个都只到「了解」。正确策略是：所有知识点达到「会用」，然后选 2~3 个你最感兴趣、或与目标岗位最相关的方向深入「精通」。

1.2 本项目的知识点分布

能力等级	知识点数量	典型内容
了解	约 40 个	各种外设原理、AI 模型结构、协议概念
会用	约 25 个	寄存器配置、串口编程、模型部署、跟随算法
精通	选 2~3 个	建议从：嵌入式底层 / 模型优化 / 控制算法 中选

1.3 如何判断自己「真的会了」

这里给三个自检问题，学完每个模块都问自己一遍：

问题	含义	举例（以串口为例）
能不用资料讲清楚吗?	检验是否真正理解，而非记忆	能讲清 UART 一帧数据是怎么传的

		吗？
能改需求吗？	检验是否掌握而非模仿	如果要改成 8E1 校验，知道改哪里吗？
能排查故障吗？	检验是否有调试能力	如果收不到数据，你的排查顺序是什么？

三个都能答上来，才算「会用」。只能答第一个，那还停留在「了解」。

第二章　知识地图

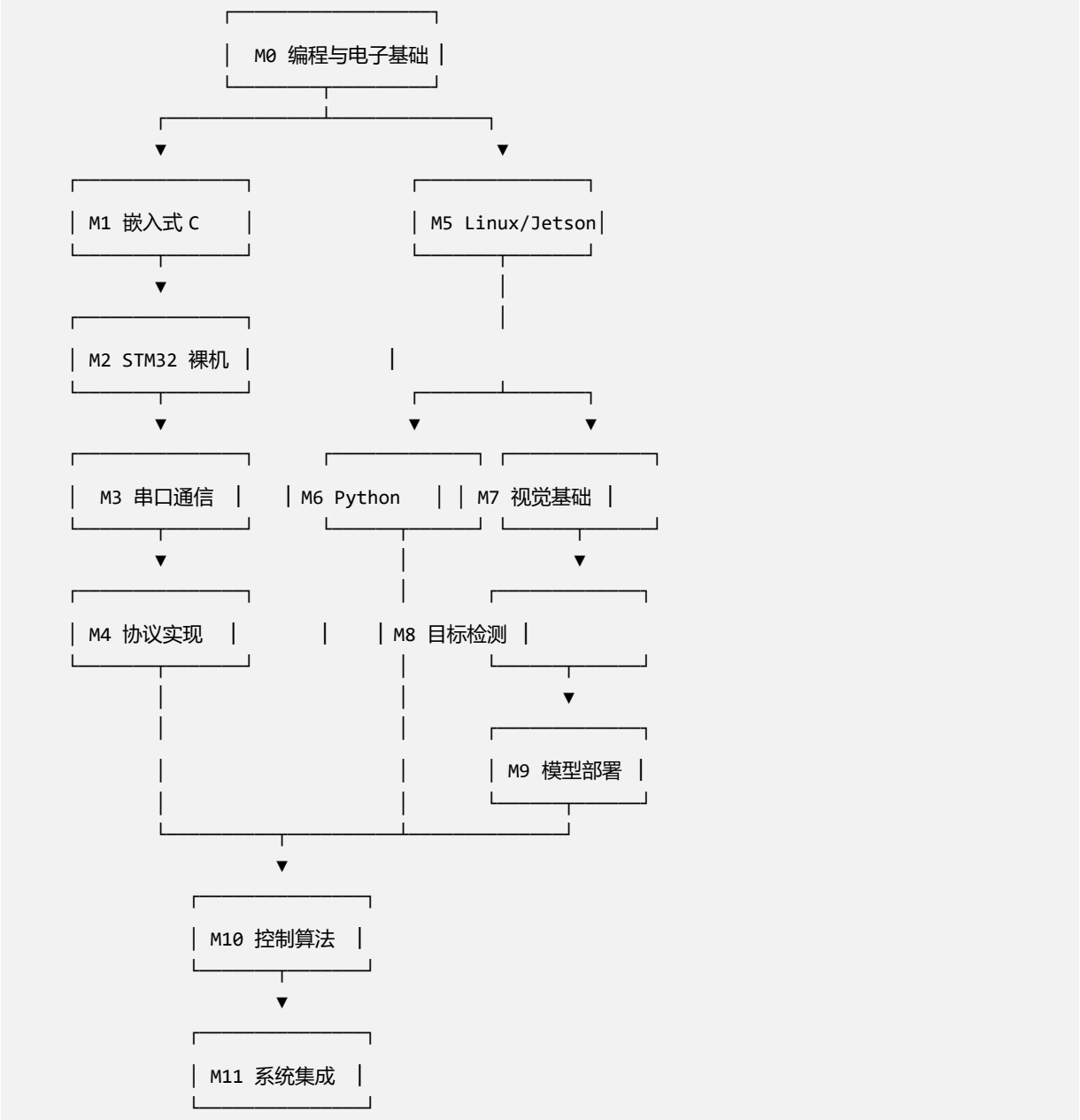
本章给出项目的完整知识结构。先看全景，再逐层展开。

2.1 12 个知识模块全景

模块	名称	核心内容	难度	依赖
M0	编程与电子基础	C 语言、Linux 命令、电路基础	⭐	无
M1	嵌入式 C 语言	位运算、指针、寄存器映射、volatile	⭐⭐	M0
M2	STM32 裸机开发	启动流程、时钟、GPIO、中断	⭐⭐⭐	M1
M3	串口通信	UART 原理、波特率、中断、缓冲	⭐⭐	M2
M4	协议设计与实现	帧格式、校验、二进制协议	⭐⭐⭐	M3
M5	Linux 与 Jetson	命令行、SSH、设备节点、权限	⭐⭐	M0
M6	Python 与串口编程	pyserial、类封装、异常处理	⭐⭐	M5
M7	计算机视觉基础	图像表示、OpenCV、GStreamer	⭐⭐	M5
M8	深度学习目标检测	YOLO 原理、推理流程、阈值	⭐⭐⭐	M7
M9	模型部署与优化	ONNX、TensorRT、FP16	⭐⭐⭐⭐	M8
M10	控制系统基础	映射、滤波、限位、开闭环	⭐⭐	M6
M11	系统集成与调试	分层调试、诊断工具、故障树	⭐⭐⭐	全部

2.2 模块依赖关系

学习顺序不是随意的，有明确的依赖关系。下图是推荐的推进路径：



2.3 两条推荐路径

根据你的目标岗位，可以选择不同的切入顺序：

路径	适合人群	推荐顺序	目标
嵌入式路线	想做 MCU / 底层开发	M0→M1→M2→M3→M4→M11	嵌入式软件工程师
AI 算法路线	想做视觉 / 部署	M0→M5→M7→M8→M9→M10	算法工程师 / 部署工程师
全栈路线（本项目）	想做完整系统	按 2.2 的依赖图顺序	系统集成 / 机器人工程师

第三章　分模块详解

本章是核心内容。每个模块包含四部分：知识点与掌握程度、项目中的对应、练习题、常见坑。

深度标注：★ = 了解　★★ = 会用　★★★ = 精通

M0 编程与电子基础

这是所有后续模块的地基。不需要很深，但必须扎实 —— 地基不牢，后面每个模块都会卡。

知识点与掌握程度

知识点	深度	说明与达标线
C 语言基本语法	★★★	指针、结构体、数组、函数、递归，能读懂 500 行程序
位运算	★★	& | ~ ^ << >>，知道如何设置/清除某一位
内存与地址	★★	知道变量在内存中的位置，理解指针就是地址
Linux 命令行	★★	cd / ls / cat / grep / chmod / ps / kill，能独立操作
SSH 远程登录	★★	能 ssh 到远程机器并执行命令
电压/电流/地	★★★	理解 GND 是参考零点，知道共地为什么必要
常见元件	★	电阻、电容、排针、杜邦线、面包板的基本用法

项目中的对应

- main.c 中的指针与结构体用法
- servo.py 中的 Python 类
- 所有远程操作都通过 SSH 完成

练习题（做完才算是「会用」）

1. 写一个函数，把整数的第 3 位设置为 1（用位运算）
2. 写一个函数，交换两个变量的值（不用临时变量）
3. 用 SSH 登录 Jetson，用 ls 和 grep 找到 /dev/tty 开头的设备

4. 解释：为什么两套电源要给舵机和单片机「共地」？

常见坑

- 位运算不熟：后面配置寄存器会完全看不懂
- 不会 Linux：所有 Jetson 操作都会卡住
- 不理解 GND：接线时容易接错，导致通信失败

M1 嵌入式 C 语言

嵌入式 C 和普通 C 的最大区别是：你要直接操作硬件地址。这需要几个特殊技巧。

知识点与掌握程度

知识点	深度	说明与达标线
寄存器映射	★★★	知道 0x40021000 这样的地址代表什么，能用结构体映射
volatile 关键字	★★★	理解为什么寄存器变量必须加 volatile
位运算配置寄存器	★★★	能用 | 置位、&~ 清位、^ 翻转
指针与强制类型转换	★★	能把整数地址转成结构体指针
宏定义	★★	用 #define 定义寄存器位
函数指针	★	知道中断向量表就是函数指针数组
内存段布局	★★	.text / .data / .bss / stack / heap 分别在哪

项目中的对应

- main.c 开头的 RCC_TypeDef / GPIO_TypeDef 结构体定义
- RCC->APB2ENR |= (1<<14) 这样的寄存器操作
- startup.c 中的中断向量表（函数指针数组）

练习题（做完才算是「会用」）

1. 手写一个结构体，映射到地址 0x40010800，包含 CRL/CRH/IDR/ODR 四个寄存器
2. 用一行代码把 GPIOA 的第 5 位设置为 1，同时不影响其他位
3. 解释 volatile 的作用，以及不加会有什么后果
4. 画出 .text / .data / .bss 在 Flash 和 RAM 中的分布

常见坑

- 不加 volatile：编译器优化会导致寄存器读写被「优化掉」
- 位运算写错：清位忘了 &~，导致影响其他位
- 结构体没加 __packed 或对齐错误：寄存器地址偏移错乱

M2 STM32 裸机开发

这是本项目最硬核的部分。不用 HAL 库，直接操作寄存器，从零构建一个可运行的工程。

知识点与掌握程度

知识点	深度	说明与达标线
MCU 启动流程	★★★	复位后从 0x08000000 读栈指针、从 0x08000004 读入口地址
中断向量表	★★	知道每个异常/中断对应的入口，能自定义
链接脚本	★★★	能读懂 MEMORY/SECTIONS，知道变量放在哪个段
时钟树	★★★	HSE→PLL→SYSCLK→AHB/APB，能算出各总线频率
GPIO 八种模式	★★★	输入浮空/上拉/下拉/模拟，输出推挽/开漏，复用推挽/开漏
USART 配置	★★★	能算出 BRR 值，知道 8N1 各参数含义
SysTick	★★	用 SysTick 实现毫秒计时
中断与 NVIC	★★	知道优先级、使能流程
GPIO 模式动态切换	★★★	本项目用它实现无电阻的舵机总线驱动

项目中的对应

- startup.c：手写的中断向量表和 Reset_Handler
- linker.ld：512K Flash + 64K RAM 的内存布局
- main.c 的 clock_init()：72MHz 时钟配置
- main.c 的 gpio_usart_init()：GPIO 和串口配置

- main.c 的 pa2_drive()：发送时推挽、发完切高阻

练习题（做完才算是「会用」）

1. 从零建一个 STM32 工程（不用 HAL），让 PA5 输出 1kHz 方波
2. 配置 USART1 为 115200 8N1，能打印 Hello
3. 用 SysTick 实现 delay_ms()，误差小于 1%
4. 解释：为什么 APB1 上的 USART2 波特率值和 APB2 上的 USART1 不同
5. 解释：PA2 为什么要「发送时推挽、发完切高阻」

常见坑

- 时钟没配对：串口波特率全错，收到乱码
- GPIO 模式选错：推挽/开漏搞混，总线冲突
- 忘了开外设时钟：配置了寄存器但外设不工作
- 链接脚本写错：程序跑飞

M3 串口通信

串口是嵌入式最常用的调试和通信手段。看似简单，但涉及大量细节。

知识点与掌握程度

知识点	深度	说明与达标线
UART 帧结构	★★★	起始位 + 数据位 + 校验位 + 停止位，能画出一帧
波特率与误差	★★★	能算 BRR 值，知道误差超过多少会失败
8N1 / 8E1	★★	知道校验位的作用，本项目舵机用 8E1（部分型号）
轮询 vs 中断	★★★	知道两种方式的优缺点和适用场景
环形缓冲区	★★★	能手写一个环形缓冲，理解读写指针
阻塞 vs 非阻塞	★★★	理解本项目第一版固件为何会丢字符
半双工 vs 全双工	★★	知道半双工需要控制发送/接收方向

项目中的对应

- USART1（PA9/PA10）：与 PC/Jetson 通信
- USART2（PA2/PA3）：与舵机总线通信
- main.c 主循环的非阻塞轮询 + 行缓冲
- servo.py 中的 pyserial 读写

练习题（做完才算是「会用」）

1. 计算 72MHz 下 115200 波特率的 BRR 值（提示：BRR = f / baud）
2. 手写一个 64 字节的环形缓冲区（put / get / available）
3. 解释：为什么主循环里加了 delay_ms(1) 就会丢字符
4. 写一个 Python 程序，打开串口发 PING，收到 PONG 就打印成功

常见坑

- 波特率算错：收到乱码
- 读数据不及时：溢出丢字节
- 没处理

  ：命令解析失败
- 半双工不控制方向：收发冲突

M4 协议设计与实现

本项目的核心难点之一。协议设计得好不好，直接决定了系统的可靠性。

知识点与掌握程度

知识点	深度	说明与达标线
文本协议 vs 二进制协议	★★★	知道各自的优缺点（可读性 vs 效率）
帧格式设计	★★★	帧头 / ID / 长度 / 指令 / 参数 / 校验和
校验和计算	★★★	能手算 XOR 校验，理解为什么需要校验
长度字段的陷阱	★★★	Length 到底包含哪些字节？本项目的最大坑
多设备寻址	★★	每个设备有 ID，支持广播

应答机制	★★	读指令需要设备返回数据
静默失败	★★★	协议错了设备通常不报错，直接丢弃

项目中的对应

- LX-16A 帧格式：55 55 ID Length Cmd Params Checksum
- servo_send_frame() 的实现
- servo_move() 的参数编码
- 固件命令解析器（文本协议）

练习题（做完才算是「会用」）

1. 手算：55 55 01 07 01 F4 01 E8 03 的校验和是多少?
2. 用官方例子验证你的实现是否产生相同的帧
3. 解释：为什么 Length 多算了 2 字节，舵机就完全不响应?
4. 设计一个简单的文本协议，支持「读温度」指令

常见坑

- 凭记忆写协议：本项目最大的坑，浪费 1 小时
- Length 计算错误：设备静默丢弃，毫无提示
- 校验和算法错误：同样静默失败
- 没考虑大小端：多字节参数顺序错

M5 Linux 与 Jetson

Jetson 是 Linux 系统，所有操作都离不开命令行。这是从 Windows 转过来的学生最容易卡住的地方。

知识点与掌握程度

知识点	深度	说明与达标线
基础命令	★★★	ls / cd / cp / mv / rm / cat / grep / find / chmod
进程管理	★★	ps / top / kill / nohup / &（后台运行）
SSH 远程登录	★★★	ssh user@ip，密钥登录
SCP 文件传输	★★★	scp 本地 远程:路径

设备节点	★★★	知道 /dev/tty* 是串口设备
权限管理	★★	知道 dialout 组才能访问串口，用 ls -l 看权限
软件包管理	★★	apt / pip 安装依赖
系统服务	★	systemctl 启停服务
设备树 overlay	★	Jetson 特有，摄像头驱动依赖它

项目中的对应

- 通过 SSH 在 Jetson 上执行所有 Python 程序
- /dev/ttyCH341USB1 就是 STM32 的串口设备
- 摄像头依赖 extlinux.conf 中的 JetsonIO overlay

练习题（做完才算是「会用」）

1. SSH 登录 Jetson，找到所有 tty 设备
2. 用 scp 把一个 .py 文件传到 Jetson 并运行
3. 查看某个串口设备的权限，并解释为什么普通用户打不开
4. 用 nohup 让一个程序在后台运行，并用 tail 看日志

常见坑

- 权限不足：串口需要 dialout 组
- 路径写错：Linux 区分大小写
- 引号被 shell 吃掉：复杂命令建议写成脚本文件
- 系统更新重置配置：extlinux.conf 被改回默认

M6 Python 与串口编程

Jetson 端的所有逻辑都用 Python 实现。Python 的易用性让快速迭代成为可能。

知识点与掌握程度

知识点	深度	说明与达标线
pyserial 基本用法	★★★	Serial() / read / write / timeout / reset_input_buffer
串口参数配置	★★	baudrate / bytesize / parity / stopbits
面向对象封装	★★★	把串口操作封装成类，暴露简洁 API
异常处理	★★★	串口打不开、超时、断开都要处理
命令行参数	★★	sys.argv 解析，支持 --frames / --seconds
时间与超时控制	★★	time.sleep / time.time() 做超时
日志与调试输出	★★	打印关键信息，过滤心跳噪声

项目中的对应

- servo.py：ServoBus 类封装了所有舵机操作
- follow.py：命令行参数解析、主循环
- detect.py：模型加载与检测函数

练习题（做完才算是「会用」）

1. 写一个类，封装「打开串口 + 发 PING + 收 PONG」
2. 给这个类加上自动识别串口的能力（遍历 /dev/tty* 探测）
3. 实现一个命令，发送后等待应答，超时返回 None
4. 给程序加上 --frames 和 --seconds 两种退出条件

常见坑

- 没设 timeout：串口读会永久阻塞
- 没清缓冲区：上次的残留数据干扰本次解析
- 异常没捕获：串口一断程序就崩溃
- 阻塞式等待应答：会导致跟随循环变慢

M7 计算机视觉基础

从「像素」到「图像」，再到「视频流」。这是进入 AI 视觉的第一步。

知识点与掌握程度

知识点	深度	说明与达标线
图像表示	★★★	知道图像是 H×W×C 的数组，BGR 与 RGB 区别
分辨率与帧率	★★★	知道 1080p / 720p 的含义，帧率与延迟的关系
OpenCV 基础	★★★	imread / resize / cvtColor / rotate / VideoCapture
GStreamer 管线	★★	知道 ! 连接各个 element，appsink 把数据交给 Python
CSI 摄像头	★★	知道 nvarguscamerasrc 走硬件 ISP
自动曝光/白平衡	★★★	理解为什么前几十帧不能用（本项目踩过的坑）
颜色空间	★★	BGR / NV12 / BGRx 的转换

项目中的对应

- camera.py：三种取帧方式（photo / video / frames）
- follow.py：GStreamer 管线 + 180° 翻转
- PREHEAT = 60 的原因

练习题（做完才算是「会用」）

1. 用 OpenCV 读一张图，打印尺寸和通道数
2. 写一个程序，打开摄像头并显示实时画面
3. 解释：为什么拍照前要预热 60~90 帧?
4. 解释：为什么 CSI 摄像头比 USB 摄像头延迟低?

常见坑

- 没预热：拍到全黑或纯噪声
- BGR/RGB 搞反：颜色异常（但检测仍能用）
- 分辨率太大：推理变慢，内存吃紧
- 摄像头装反：画面上下颠倒

M8 深度学习目标检测

从「这是什么」到「它在哪里」。目标检测是机器人视觉的核心任务。

知识点与掌握程度

知识点	深度	说明与达标线
检测任务定义	★★★	分类（是什么）+ 定位（在哪里）
YOLO 原理	★★	网格划分、anchor、置信度、NMS 后处理
模型规模与速度	★★★	n/s/m/l/x 五种规模，参数越多越慢越准
COCO 数据集	★★	80 类常见物体，person 是第 0 类
推理流程	★★★	预处理（resize/归一化）→ 推理 → 后处理（NMS）
置信度阈值	★★★	阈值越高越严格，漏检增加
IOU 与 NMS	★★	知道 NMS 用来去除重叠框

项目中的对应

- detect.py：模型加载、检测、结果结构化
- follow.py：筛选 person 类，取面积最大者
- CONF = 0.4 的由来

练习题（做完才算是「会用」）

1. 用 YOLOv8n 检测一张图，打印所有检测结果
2. 调整 conf 从 0.1 到 0.9，观察检测数量变化
3. 解释：为什么跟随算法只取「面积最大的 person」？
4. 解释：什么是 NMS，为什么需要它?

常见坑

- 模型太大：帧率不够，跟随有延迟
- 阈值太高：人走远了就检测不到
- 没过滤类别：把椅子也当目标
- 输入尺寸和训练尺寸不一致：精度下降

M9 模型部署与推理优化

训练好的模型只是开始，能不能在边缘设备上跑得动才是关键。这是 AI 落地的核心技能。

知识点与掌握程度

知识点	深度	说明与达标线
PyTorch → ONNX	★★★	知道为什么要转 ONNX（跨框架中间格式）
ONNX → TensorRT	★★★	用 trtexec 或 Python API 构建引擎
精度模式	★★★	FP32 / FP16 / INT8 的速度与精度权衡
算子融合	★★	TensorRT 会自动合并部分算子
kernel 自动调优	★★	针对目标 GPU 选择最优实现
性能瓶颈分析	★★★	区分预处理瓶颈 vs 推理瓶颈
显存管理	★★	知道模型占用多少显存，避免 OOM
引擎元数据丢失	★★	trtexec 导出的引擎不含类别名，需要手动映射

项目中的对应

- YOLOv8n.pt → onnx → tensorrt engine（8.82MB）
- 640×640 输入时 27.3ms → 19.6ms（1.39 倍加速）
- 720 输入时无明显提升（预处理瓶颈）

练习题（做完才算是「会用」）

1. 把一个 PyTorch 模型导出为 ONNX
2. 用 trtexec 构建 TensorRT 引擎，并记录耗时
3. 对比 FP32 和 FP16 的推理速度
4. 解释：为什么 640 输入加速明显，720 输入不明显？

常见坑

- 引擎构建很慢（几分钟）：这是正常的，一次构建多次使用
- 精度掉太多：FP16 通常没问题，INT8 需要校准
- 输入尺寸不匹配：引擎报错

- 显存不够：多个模型同时加载导致 OOM

M10 控制系统基础

把「检测结果」变成「机械动作」，中间需要控制算法。这是很多人忽略但很重要的一环。

知识点与掌握程度

知识点	深度	说明与达标线
开环 vs 闭环	★★★	本项目的舵机是开环（发命令，不反馈调节）
位置映射	★★★	把画面坐标线性映射到舵机角度
指数平滑滤波	★★★	pan = pan + (target - pan) × α，能推导稳态误差
限位保护	★★★	软件限制运动范围，保护机械结构
死区	★★	小误差不动作，避免抖动
采样率与控制周期	★★	知道控制频率受什么影响
超调与振荡	★★	知道 α 太大会振荡，太小会滞后

项目中的对应

- follow.py 的偏移映射公式
- SMOOTH = 0.25 的选择
- PAN_MIN/PAN_MAX = 400/600 的安全限位

练习题（做完才算是「会用」）

1. 实现指数平滑函数，输入一组阶跃数据，画出响应曲线
2. 把 SMOOTH 从 0.1 调到 1.0，观察跟随效果变化
3. 设计一个限位函数，超限时返回边界值并给出提示
4. 解释：为什么需要平滑？直接到位会有什么问题？

常见坑

- α 太大：舵机抖动、异响
- α 太小：跟随滞后，人走远了才转
- 没限位：舵机顶到机械限位，烧电机或断排线

- 控制频率太低：跟不上快速移动

M11 系统集成与调试

把各个模块拼起来，并让它在出问题时能快速定位 —— 这是工程师最核心的能力。

知识点与掌握程度

知识点	深度	说明与达标线
分层调试	★★★	从下往上：电源→通信→协议→算法→应用
诊断工具设计	★★★	在固件里加自测命令，比示波器更快
日志与回显	★★★	关键节点打印，方便定位
故障树	★★★	把「不工作」拆解成可验证的子问题
边界条件	★★★	位置 0/1000、断线、断电恢复
跨平台联调	★★	Windows 开发 + Linux 运行
版本管理	★★★	Git 提交、分支、回滚

项目中的对应

- LOOP / SCAN / BAUD 三个诊断命令
- 本项目 8 个难点的排查过程
- Git 提交历史

练习题（做完才算是「会用」）

1. 给固件加一个「读温度」的诊断命令
2. 画一张「舵机不转」的故障树（至少 5 个分支）
3. 复现一个 bug（比如故意插错线），然后按故障树定位
4. 用 Git 提交你的修改，并写清楚 commit message

常见坑

- 一上来就改代码：应该先定位问题
- 没有诊断手段：只能靠猜
- 不写文档：过一周自己都忘了怎么搭的
- 不版本管理：改坏了回不去

第四章　学习路线

按阶段推进，每个阶段有明确的产出。建议每天 2~3 小时，总共 8~12 周。

4.1 五阶段路线

阶段	周期	模块	产出（必须能拿出来）
阶段一 打基础	第 1~2 周	M0, M1	能手写位运算、能 SSH 登录、能解释共地
阶段二 嵌入式入门	第 3~4 周	M2, M3	从零建 STM32 工程、串口能收发
阶段三 协议与通信	第 5~6 周	M4, M5, M6	能实现一个二进制协议、Python 能控制设备
阶段四 视觉与 AI	第 7~9 周	M7, M8, M9	能部署 YOLO 到 Jetson、能达到 30 FPS
阶段五 系统集成	第 10~12 周	M10, M11	完成一个完整闭环项目、写文档

4.2 每周检查点

周	学习内容	检查点（做不到就复习）
第 1 周	C 语言位运算、指针、结构体；Linux 基础命令	能写位操作函数、能用命令行完成日常操作
第 2 周	寄存器映射、volatile、内存段布局	能解释结构体映射寄存器的原理
第 3 周	STM32 启动流程、时钟树、GPIO	能让一个 LED 闪烁（不用库）
第 4 周	USART 配置、中断、环形缓冲	能串口打印 Hello 并接收命令
第 5 周	协议设计、校验和、帧格式	能实现一个带校验和的协议
第 6 周	Jetson 操作、Python 串口编程	能用 Python 控制一个串口设备
第 7 周	OpenCV 基础、摄像头采集	能打开摄像头并保存图片
第 8 周	YOLO 原理、模型推理	能检测一张图里的物体
第 9 周	ONNX/TensorRT 部署、性能优化	能把模型跑到 30 FPS 以上
第 10 周	控制算法、映射、滤波	能实现一个平滑的跟随算法
第 11 周	系统集成、调试方法	能把各模块串起来跑通

第 12 周	文档、复盘、GitHub 发布	能写出完整项目文档并开源

4.3 时间分配建议

活动	时间占比	说明
看资料 / 视频	20%	不要只看，要边看边敲
动手写代码	50%	这是最重要的部分
调试排错	20%	排错才是最锻炼能力的
写文档 / 复盘	10%	不写文档等于没学到

很多学生的误区是「看视频 80% 时间，写代码 20%」。正确的比例应该反过来。

第五章　能力自评

学完每个阶段，用下面的表格自评。2 分 = 不会，5 分 = 熟练。

5.1 分模块自评表

模块	自评项	1~5 分
M0	能用位运算设置/清除某一位	
M0	能 SSH 登录并用命令行完成日常操作	
M0	能解释「共地」的必要性	
M1	能写一个结构体映射到指定内存地址	
M1	能解释 volatile 的作用	
M2	能说出 STM32 上电后的执行顺序	
M2	能算出 USART 的 BRR 值	
M2	能解释 GPIO 八种模式的区别	
M3	能画出一帧 UART 数据	
M3	能手写环形缓冲区	
M4	能手算协议校验和	
M4	能解释 Length 字段的正确算法	
M5	能找到并配置串口设备	
M5	能用 nohup 后台运行程序	
M6	能封装一个串口类	
M6	能处理串口超时和断开	
M7	能用 OpenCV 读图和处理	
M7	能解释为什么要预热帧	
M8	能解释 YOLO 的推理流程	
M8	能调整置信度阈值并理解影响	
M9	能导出 ONNX 并构建 TensorRT 引擎	

M9	能分析推理瓶颈在哪	
M10	能实现指数平滑滤波	
M10	能设计限位保护	
M11	能画故障树并定位问题	
M11	能用 Git 管理代码	

5.2 达标标准

总分	评价	建议
0 ~ 40	入门阶段	继续按路线学习，重点补基础
41 ~ 70	基础扎实	可以尝试做完整项目
71 ~ 95	熟练掌握	可以投递实习岗位
96 以上	优秀	可以尝试独立负责模块

第六章　岗位对应

学完这些知识，可以对应哪些工作？下面列出相关岗位及要求。

岗位	核心要求	本项目的对应	需要补的
嵌入式软件工程师	C 语言、MCU 外设、RTOS、通信协议	M1~M4 完全对应	RTOS、CAN、USB
边缘 AI 部署工程师	模型转换、TensorRT/ONNX、性能优化	M7~M9 完全对应	INT8 量化、多模型调度
机器人软件工程师	ROS、运动控制、传感器融合	M10~M11 对应	ROS2、SLAM、路径规划
计算机视觉工程师	深度学习、OpenCV、模型训练	M7~M8 对应	模型训练、数据标注
系统集成工程师	跨平台调试、系统架构	M11 完全对应	产品化经验

这份项目经历在简历上怎么写？

项目名称：边缘 AI 云台跟随系统

担任角色：全栈开发（个人独立完成）

- 基于 Jetson Orin Nano + STM32F103 实现实时视觉跟随，端到端延迟 < 50ms，检测帧率 37 FPS
- 从零编写 STM32 裸机固件（4.5KB，无 HAL 依赖），实现 LX-16A 总线舵机协议（含 ID 管理、位置读回）
- 部署 YOLOv8n 到 TensorRT FP16，实现 1.39 倍加速
- 设计安全限位与平滑滤波算法，定位精度 ±0.24°
- 开发自动化烧录工具与诊断命令，提升调试效率

第七章　常见误区

下面是学生在这个领域最容易走偏的 10 个地方。

#	误区	正确做法
1	看视频代替写代码	看懂了≠会写。必须动手，哪怕照着敲一遍。
2	直接抄代码不理解	项目能跑，但换个需求就不会改。要理解每一行在做什么。
3	跳过基础直接做项目	没有 C 语言和 Linux 基础，后面每一步都会卡。
4	追求「一次成功」	真正的工程是不断试错。本项目的 8 个难点，每个都花了几小时。
5	不写文档	过一周自己都忘了怎么搭的。写文档是最好的复习。
6	不做版本管理	改坏了回不去。Git 是必备技能。
7	只学「能做」的部分	调试、文档、测试同样是能力，找工作面试会问。
8	害怕底层	寄存器、汇编看着吓人，但其实是确定性的知识，学会了就是学会了。
9	不总结	做完项目不复盘，等于白做。要问自己「我学到了什么」。
10	拿别人的项目当自己的	面试官一问细节就露馅。要自己做一遍。

附录 A：术语表

缩写	全称与含义
MCU	Microcontroller Unit，单片机
HAL	Hardware Abstraction Layer，硬件抽象层（ST 提供的库）
GPIO	General Purpose Input/Output，通用输入输出口
UART	Universal Asynchronous Receiver/Transmitter，通用异步收发器
BRR	Baud Rate Register，波特率寄存器
ISR	Interrupt Service Routine，中断服务程序
NVIC	Nested Vectored Interrupt Controller，嵌套向量中断控制器
DMA	Direct Memory Access，直接内存访问
FPC	Flexible Printed Circuit，柔性印刷电路（排线）
CSI	Camera Serial Interface，摄像头串行接口
FPS	Frames Per Second，每秒帧数
NMS	Non-Maximum Suppression，非极大值抑制
IOU	Intersection over Union，交并比
ONNX	Open Neural Network Exchange，开放神经网络交换格式
TensorRT	NVIDIA 的推理优化库
FP16/FP32/INT8	不同精度的数值表示
OOM	Out Of Memory，显存/内存不足
IDE	Integrated Development Environment，集成开发环境

附录 B：推荐学习资料

领域	资料
STM32	《STM32 库开发实战指南》野火（配套指南者开发板）
STM32	STM32F103 Reference Manual（RM0008）官方手册

STM32	ARM Cortex-M3 权威指南（Joseph Yiu）
嵌入式	《嵌入式 C 语言编程修炼之道》
Linux	《鸟哥的 Linux 私房菜》
Python	《Python 编程：从入门到实践》
OpenCV	OpenCV 官方文档 + 《学习 OpenCV 3》
深度学习	《深度学习入门：基于 Python 的理论与实现》（鱼书）
YOLO	Ultralytics 官方文档
TensorRT	NVIDIA TensorRT Developer Guide
Jetson	NVIDIA Jetson 官方文档 + JetsonHacks 项目
项目	github.com/ZELDA-LINK-F/edge-ai-gimbal（本项目源码）
资料	github.com/ZELDA-LINK-F/edge-ai-learning（配套资料库）

附录 C：项目文件索引

文件	说明	对应模块
stm32/main.c	主程序：时钟、串口、命令解析、舵机协议	M1~M4
stm32/startup.c	启动文件：中断向量表、Reset_Handler	M2
stm32/linker.ld	链接脚本：内存布局	M1, M2
stm32/flash.ps1	自动烧录脚本	M11
jetson/camera.py	摄像头采集（三种模式）	M7
jetson/detect.py	YOLO 检测	M8, M9
jetson/servo.py	舵机控制库（含安全限位）	M6, M10
jetson/follow.py	云台跟随主程序	M10, M11
docs/01~08	8 篇技术文档	全部

— 指南完 —

记住：项目只是载体，真正值钱的是你在解决问题过程中建立的知识体系和思维方式。